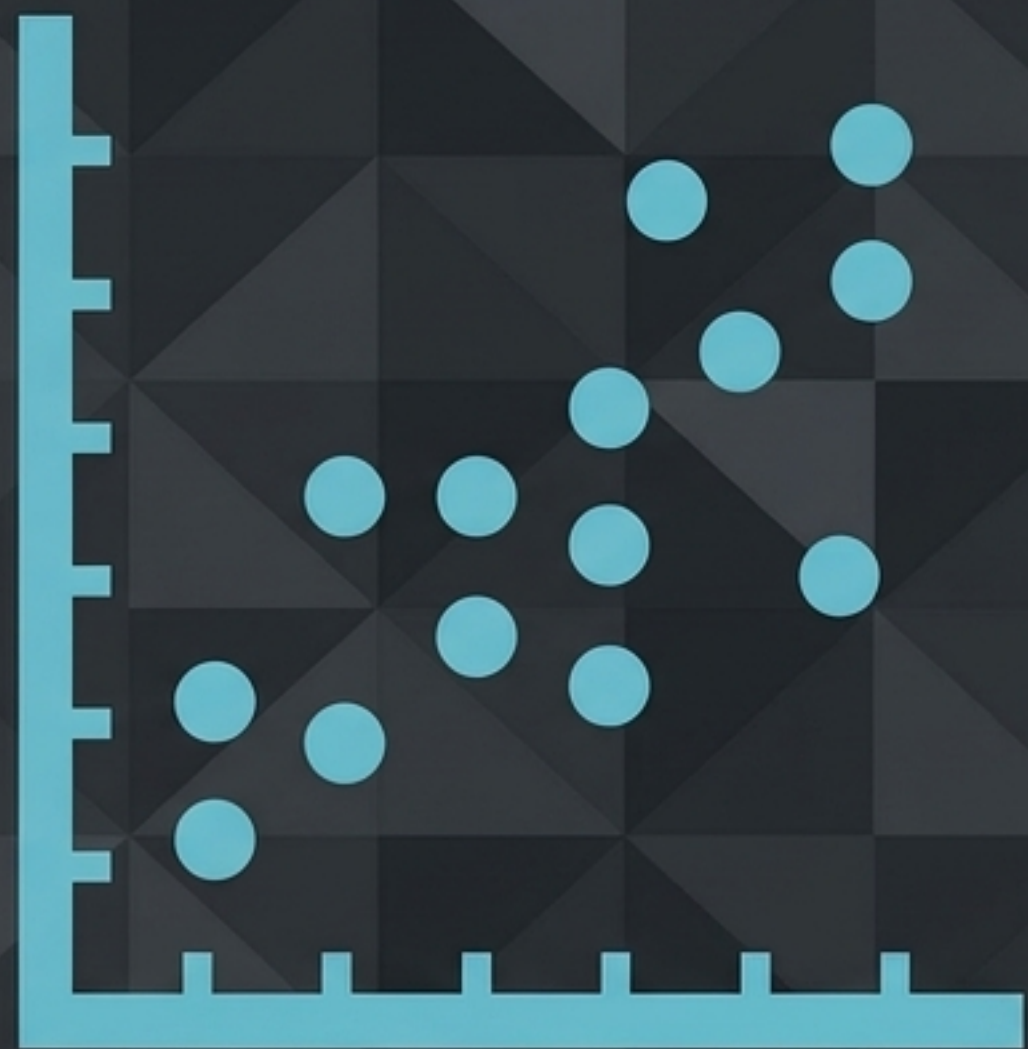
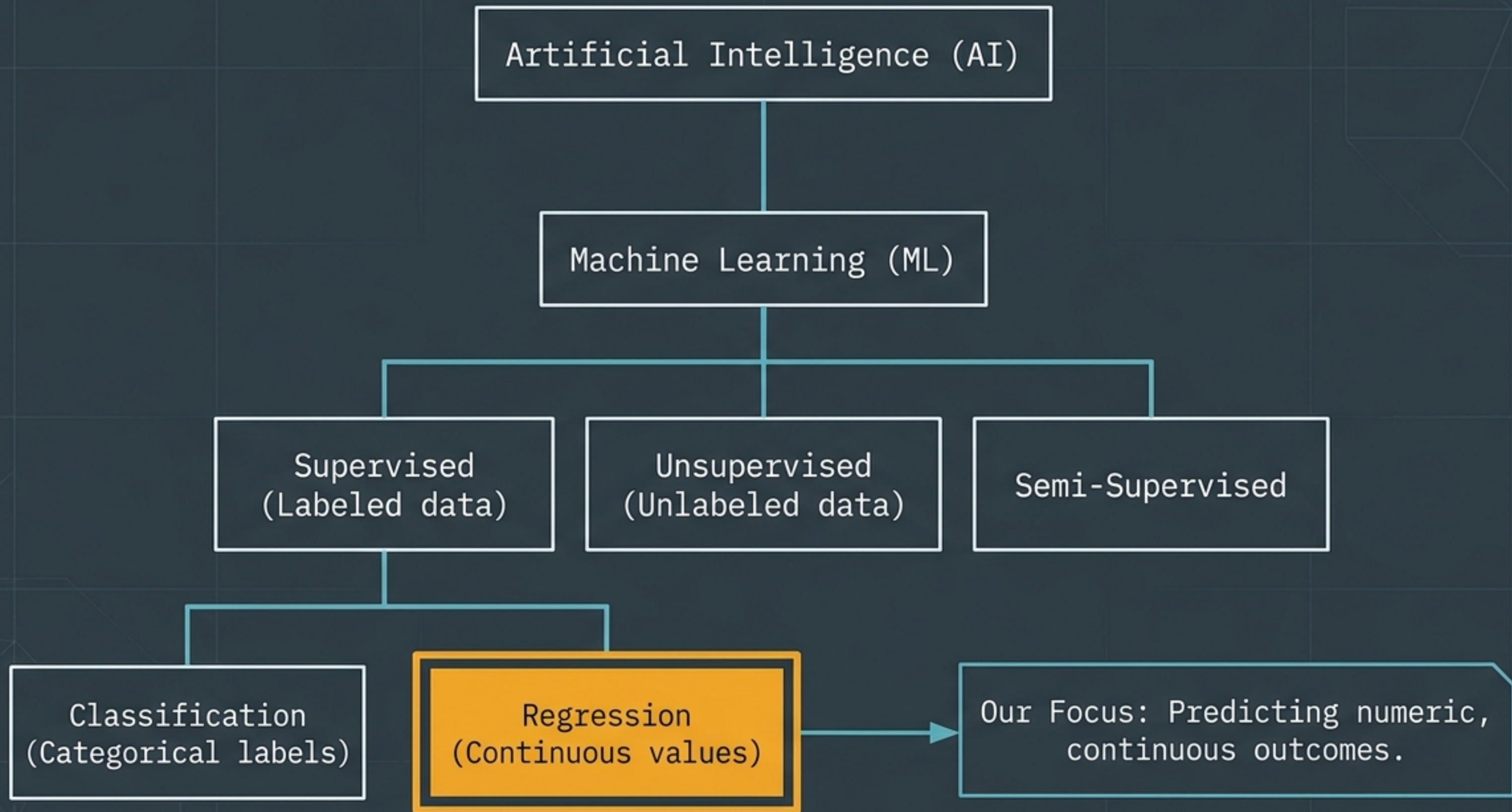




PART 3 OF THE
DATA SCIENCE IN
PYTHON SERIES

THE ARCHITECT'S GUIDE TO REGRESSION

Predicting Continuous
Realities: From Simple Lines
to Regularized Hyperplanes



Anatomy of a Prediction



Regression models the relationship between explanatory variables and a continuous target variable.

Carat	Cut	Color	Clarity	Price
0.23	Ideal	E	SI2	326
0.21	Premium	E	SI1	326
0.23	Good	E	VS1	327
0.29	Premium	I	VS2	334

The Features (X):
The inputs or
predictors.

The Target (Y):
The continuous
variable we want
to predict.

Goal 1: Prediction



Applying models to new, unobserved data to forecast future values (e.g., forecasting tomorrow's sales or a diamond's future price).

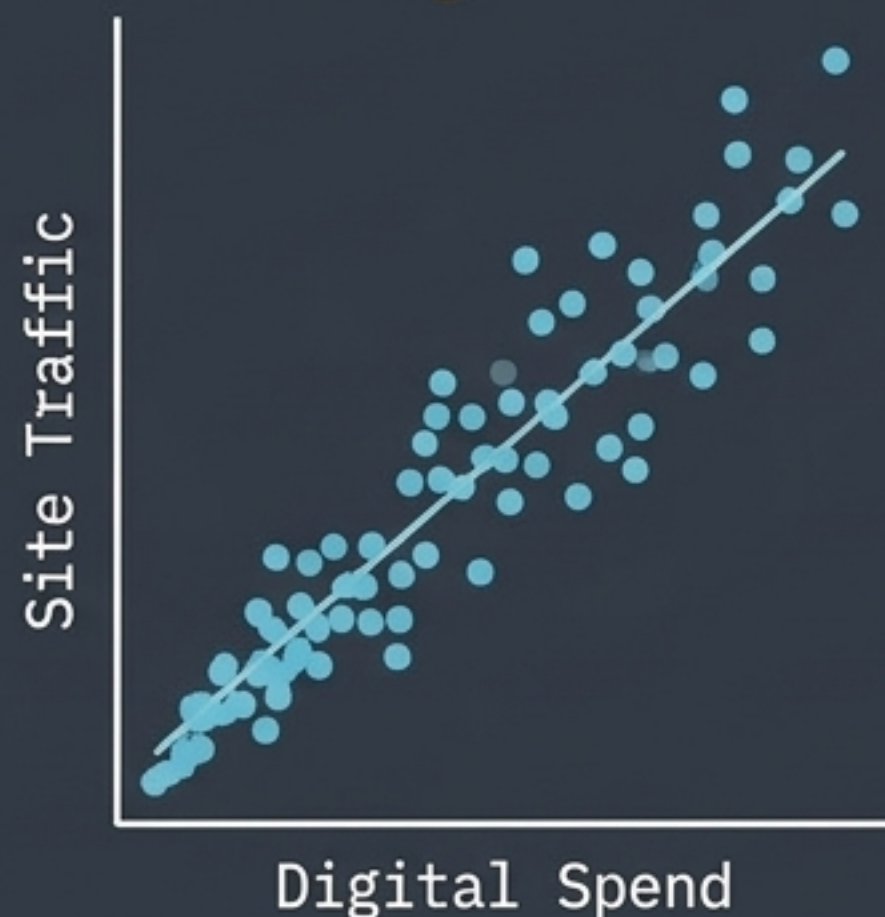
Goal 2: Inference



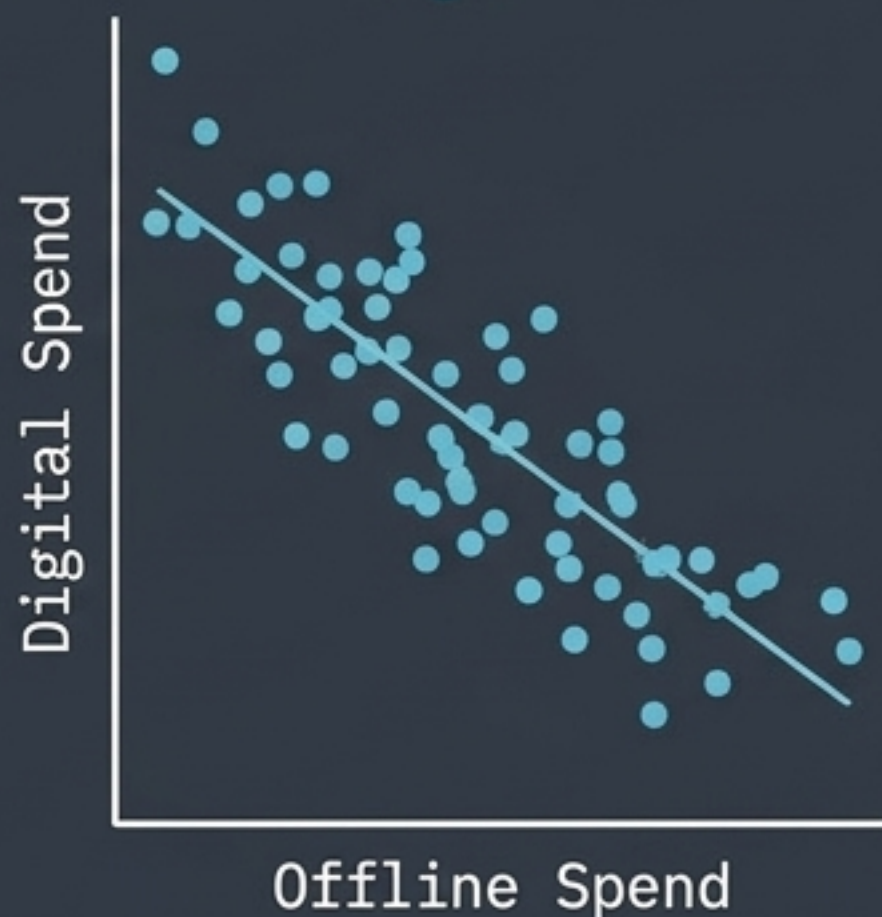
Understanding the relationship and isolated impact of individual features on the target (e.g., Exactly how much does adding one extra carat increase the price?).

EDA: Visualizing Linear Relationships

Positive Relationship:
Variables move in the
same direction.



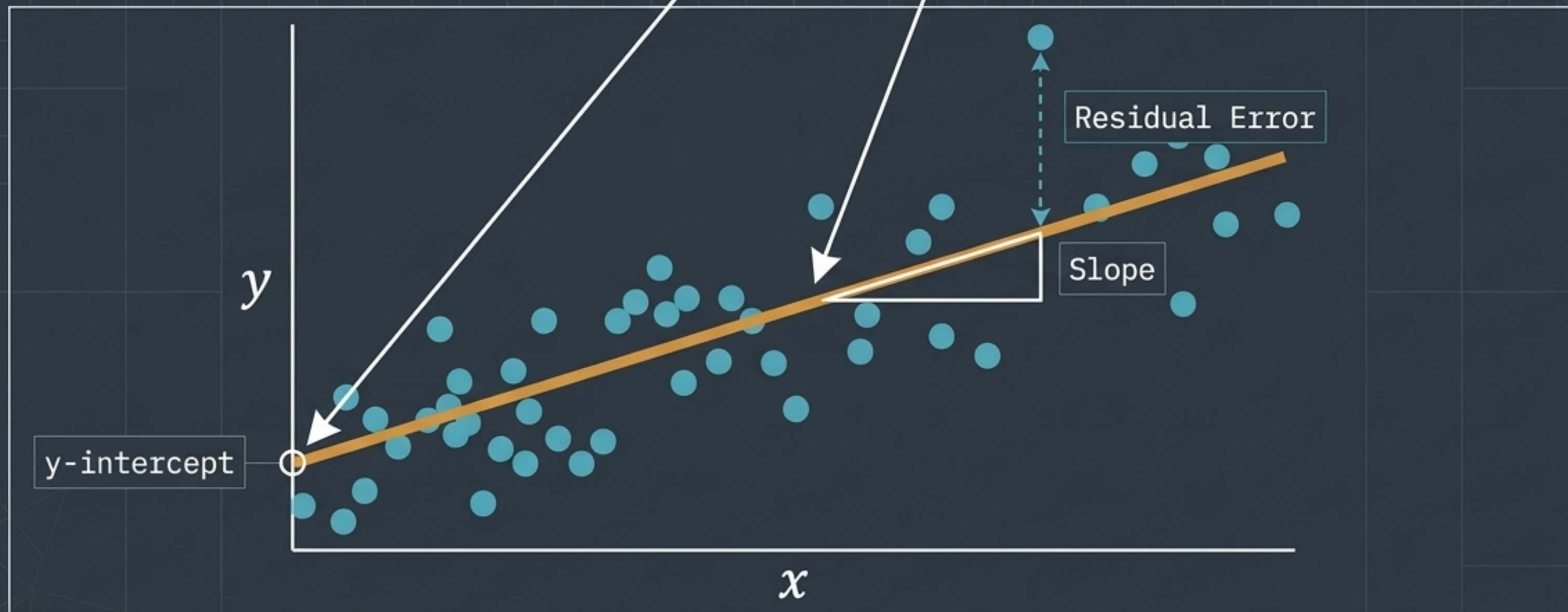
Negative Relationship:
Variables move in
opposite directions.



No Relationship:
Randomly scattered;
no association.



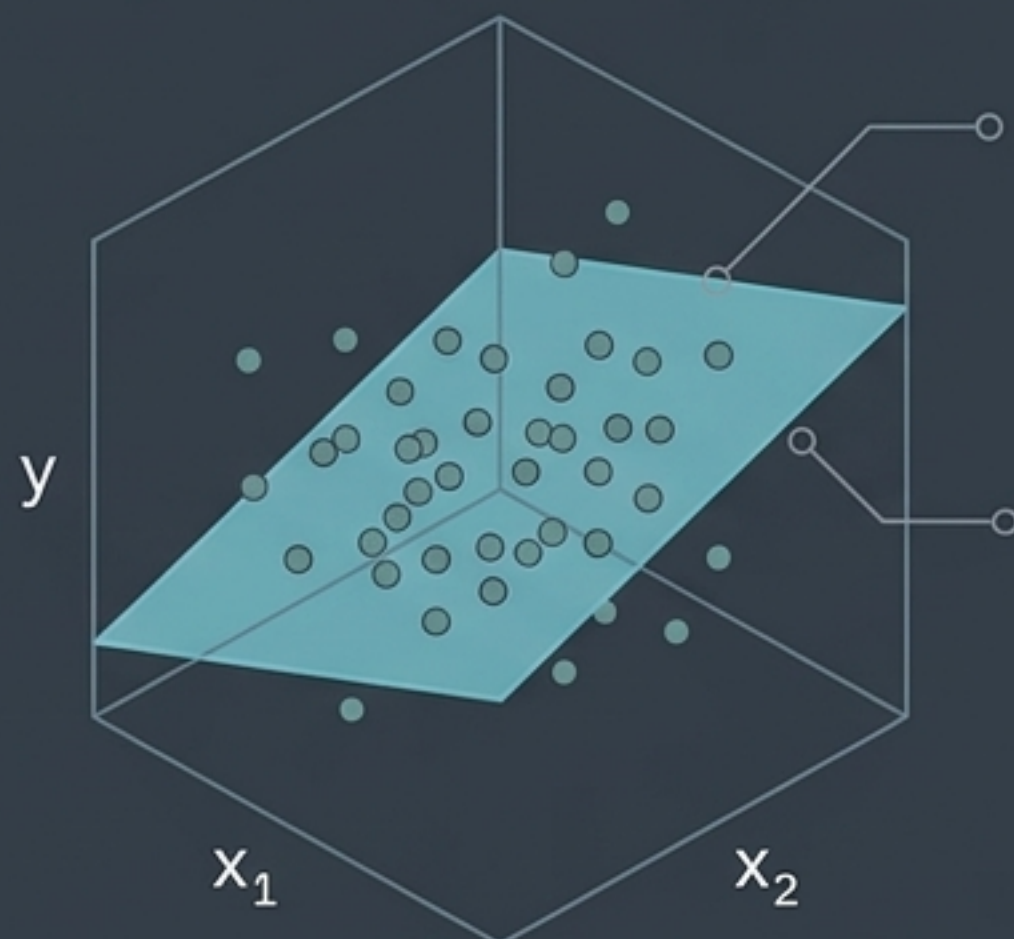
$$\hat{y} = \theta_0 + \theta_1 x$$



Ordinary Least Squares (OLS) – The model finds the best-fit line by minimizing the Mean Squared Error (MSE), perfectly balancing the pull of all data points.

Extending the Model: Multiple & Polynomial Regression

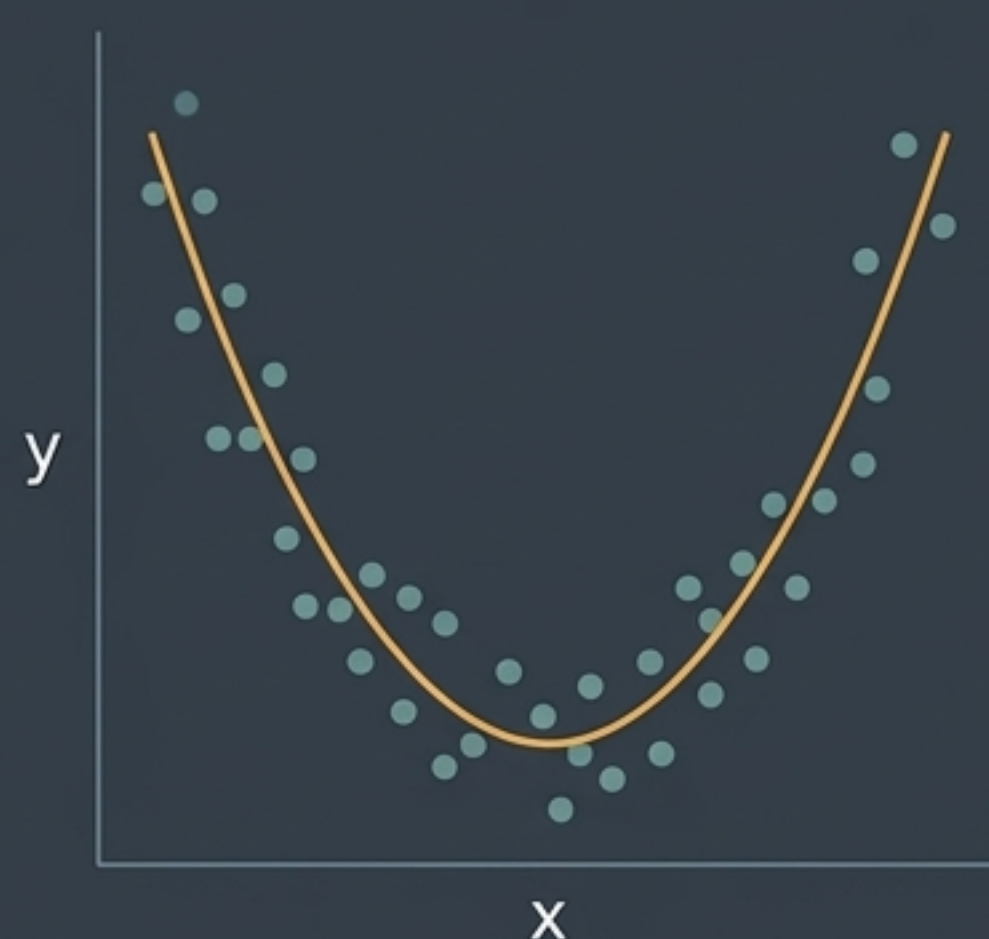
Multiple Regression



Multiple Linear Regression

Multiple Linear Regression: Uses 2+ independent variables. The model upgrades from a simple 2D line to a multidimensional hyperplane.

Polynomial Regression



Polynomial Regression

Polynomial Regression: Transforms features into polynomial terms (e.g., quadratic, cubic) to model complex curves while keeping the underlying parameters linear.

The Core Threat: Overfitting & Variance

High Variance (Overfitting)



The model memorizes training noise rather than the true pattern. It will fail spectacularly on unseen data.

Balanced Tradeoff



The optimal model accepts a small amount of bias to dramatically improve generalization.

Regularized Regression (The Cure for Overfitting)

Total Cost Function: What the model wants to minimize.

$$J = SSE + \alpha R$$

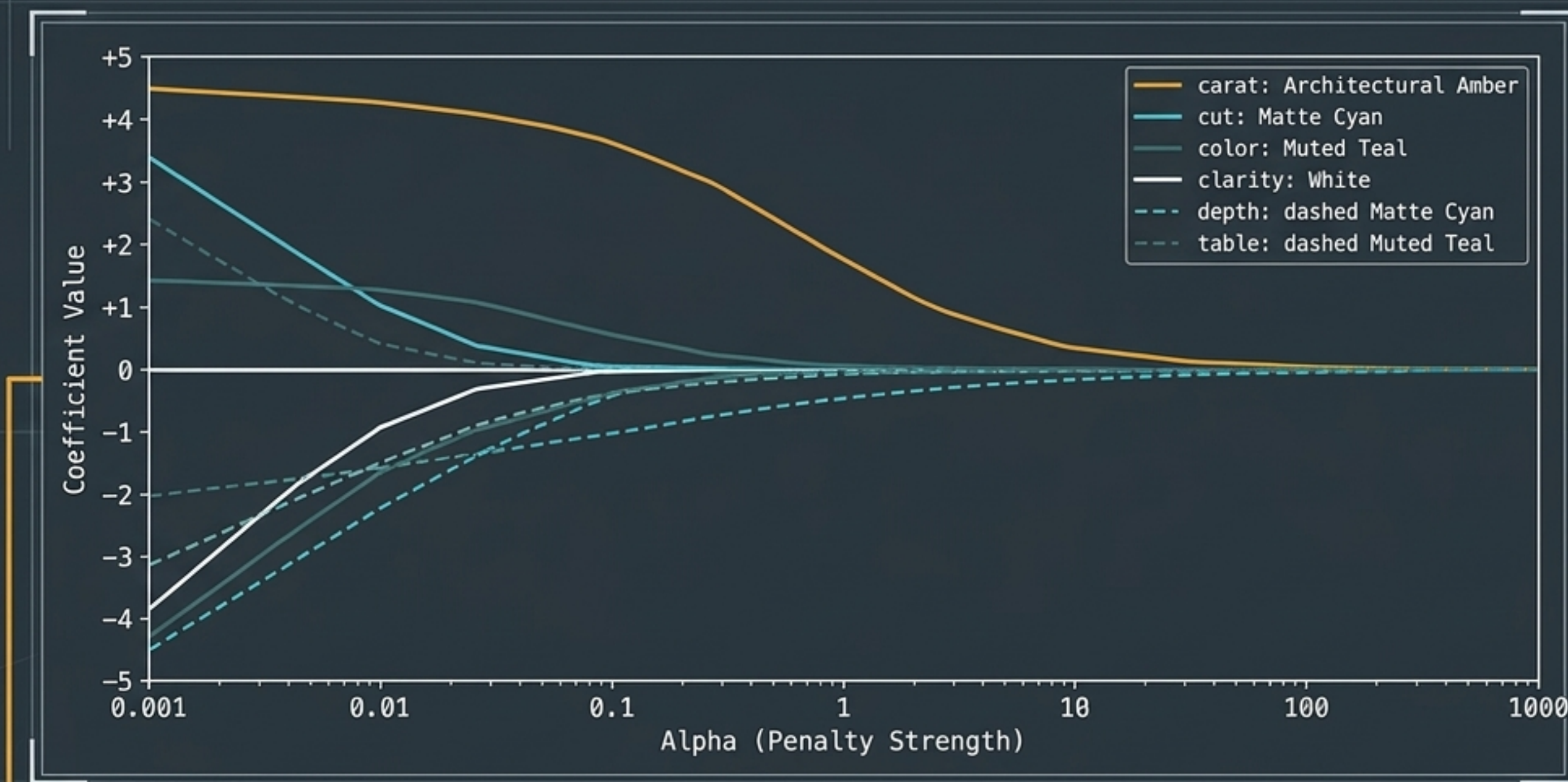
Sum of Squared Errors: The traditional OLS fit.

The Penalty Term: This deliberately punishes the model for being too complex. It forces the model to artificially add bias and avoid the absolute line of best fit, ensuring better real-world performance.

Regularized Regression Comparison: Ridge, Lasso, & Elastic Net

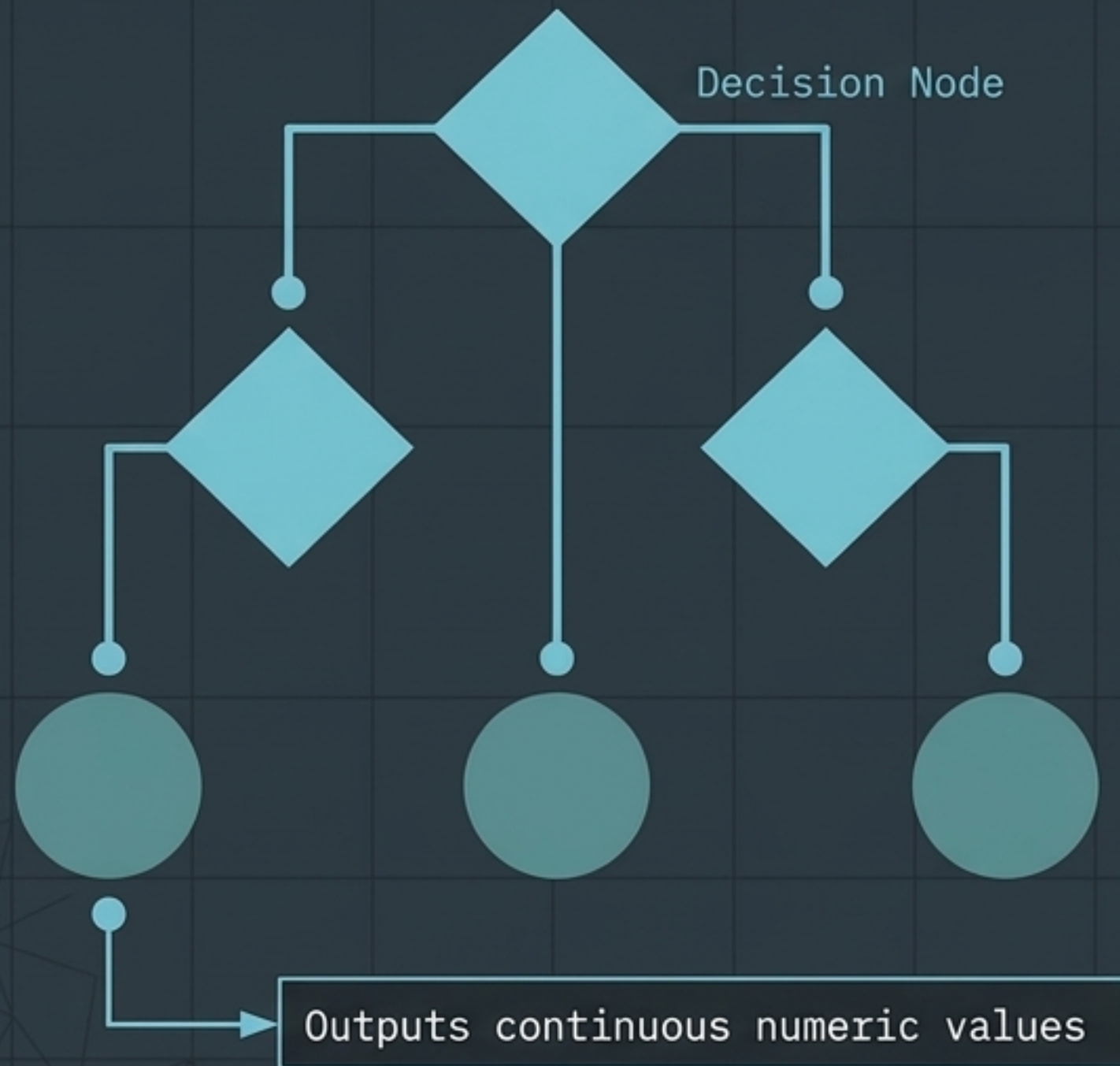
	Ridge Regression	Lasso Regression	Elastic Net
Penalty Type	L2 (Sum of squared coefficients)	L1 (Sum of absolute coefficients)	Combination of L1 and L2 terms
Effect on Coefficients	Shrinks coefficients toward zero, but never fully reaches it.	Can drop coefficients completely to zero.	Balances shrinkage and outright dropping.
Best Use Case	Datasets with highly correlated features.	Automatic Feature Selection (simplifying complex models).	Highly complex datasets that require both behaviors.

Visualizing Coefficient Shrinkage (The Power of Alpha)



Notice the decay: As the Alpha penalty increases, feature weights are forcibly shrunk toward zero. Some decay immediately, while others (like carat) remain resilient, proving their strong predictive power.

Modern Alternatives: Tree-Based Regression



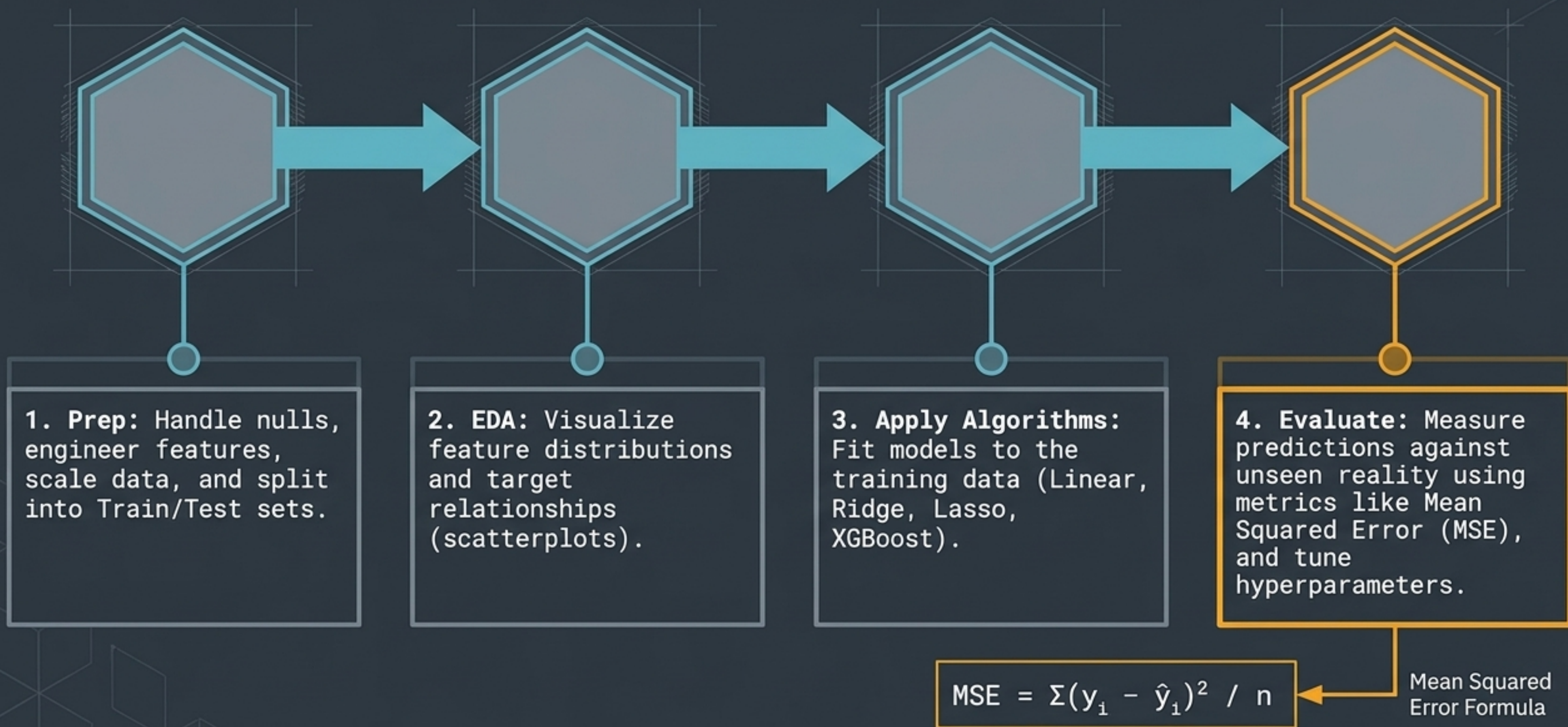
Decision Tree Regressors:

Handle non-linear data by recursively splitting the dataset based on feature values to maximize information gain.

Ensemble Methods:

Advanced algorithms like Random Forest (bagging multiple trees) and XGBoost (gradient boosting errors sequentially) drastically improve accuracy and manage the bias/variance tradeoff without strictly linear constraints.

The Regression Modeling Workflow

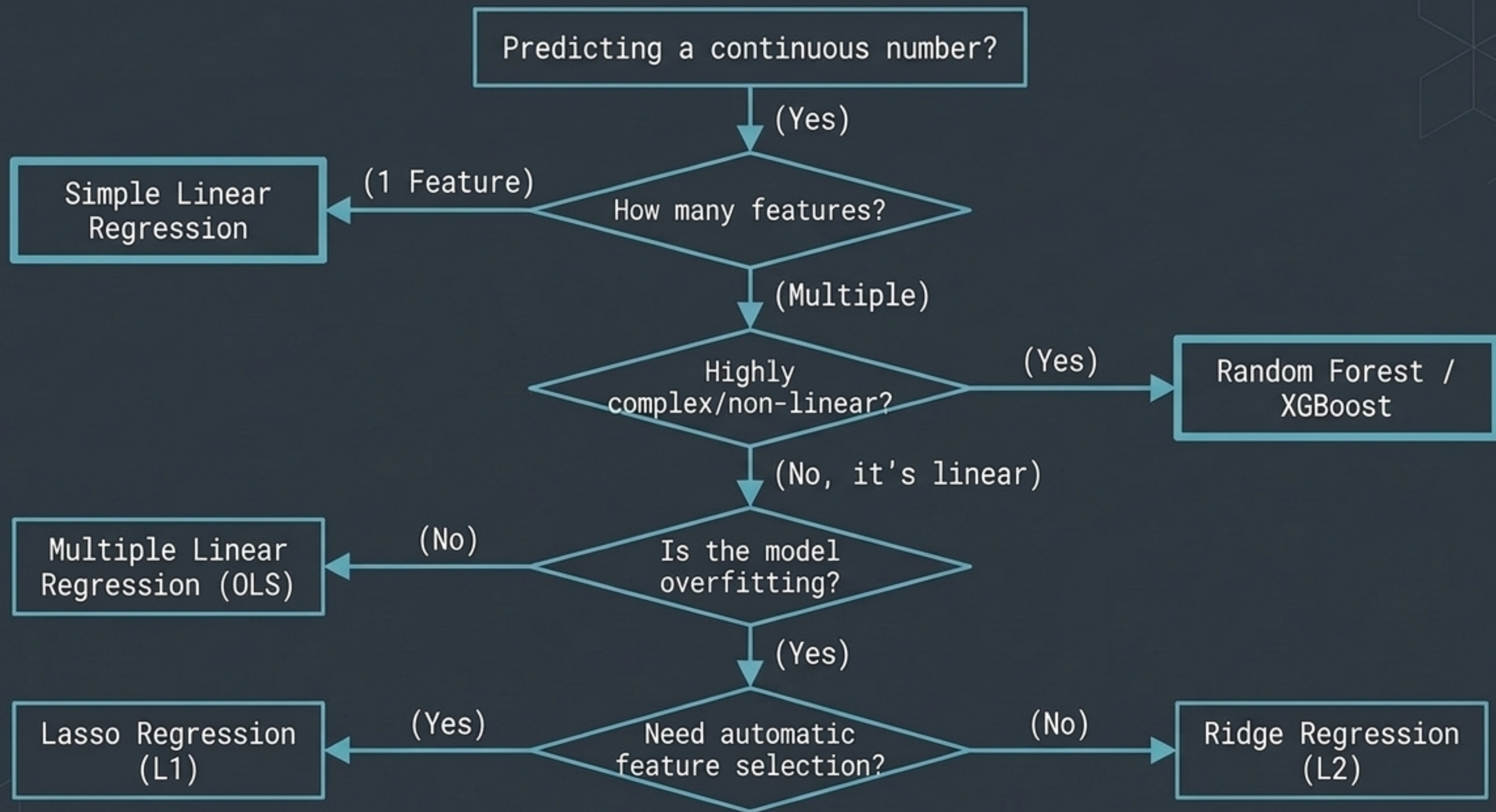


Python Tooling Cheat Sheet

```
# Modeling Imports
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor
import xgboost as xgb

# Evaluation Imports
from sklearn.metrics import mean_squared_error
```

Scikit-learn (traditional models) and XGBoost provide the foundational libraries for rapid model instantiation, fitting, and evaluation.



The Regression Decision Tree: Use this diagnostic framework to match the architecture to the dataset.